

Back-of-the-Envelope Estimation

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

English:

Before you can design a system correctly, you need a rough sense of its scale -- is this 100 requests per second or 100,000? Back-of-the-envelope estimation is the skill of quickly, roughly calculating these numbers using simple arithmetic.

Hindi:

किसी system को सही तरीके से design करने से पहले, आपको उसके scale का एक मोटा अंदाज़ा चाहिए -- क्या यह 100 requests per second है या 100,000? Back-of-the-envelope estimation वह skill है जिससे आप साधारण arithmetic से जल्दी, मोटा-मोटा calculate कर लेते हैं।

Important Words:

- **Arithmetic** – अंकगणति

Backend engineer analogy

English:

This is the same instinct you use when deciding whether a cron job report needs to process a few thousand rows (just loop in PHP) versus 50 million rows (needs chunking, queues). HLD interviews just ask you to do this sizing exercise out loud.

Hindi:

यह वही instinct है जो आप तब use करते हैं जब यह decide करते हैं कि किसी cron job report को कुछ हजार rows process करने हैं (बस PHP में loop करें) या 5 करोड़ rows (chunking, queues चाहिए)। HLD interviews बस आपसे यही sizing exercise ज़ोर से बोलकर करने को कहते हैं।

Important Words:

- **Sizing (figurative)** – आकार का अंदाज़ा लगाना

The core method

English:

It's worth being honest about why this skill feels uncomfortable at first: most backend work rewards precision. Estimation deliberately asks you to be comfortable being roughly right, fast, in front of someone watching.

Hindi:

यह ईमानदारी से समझना ज़रूरी है कि यह skill शुरुआत में असहज क्यों लगती है: ज्यादातर backend काम precision को महत्व देता है। Estimation जानबूझकर आपसे कहता है कि आप जल्दी, किसी के सामने, थोड़ा गलत होकर भी सही रहने में सहज रहें।

Important Words:

- **Precision** – सटीकता

English:

- Clarify the metric you actually need -- requests per second? Storage per day?
- Break it into a simple formula.
- Plug in round numbers -- use 10, 100, 1000, 1 million; never chase false precision.
- Do the arithmetic step by step, out loud.
- Sanity-check and use the result -- state what the number implies for your design.

Hindi:

- यह साफ करें कि आपको असल में कौन सा metric चाहिए -- requests per second? storage per day?
- इसे एक साधारण formula में तोड़ें।
- round numbers use करें -- 10, 100, 1000, 1 million; झूठी precision के पीछे कभी मत भागें।
- arithmetic को step by step, ज़ोर से बोलकर करें।
- result को sanity-check करें और use करें -- बताएं कि यह number आपके design के लिए क्या मतलब रखता है।

Important Words:

- **Sanity-check** – सैनटी-चेक / जांच लेना

English:

Beyond QPS and storage, a third estimate worth practicing is bandwidth -- how much data flows in/out per second. A service doing 1,000 requests/second with a 5KB average response needs roughly 5MB/second of outbound bandwidth.

Hindi:

QPS और storage के अलावा, practice करने लायक एक तीसरा estimate bandwidth है -- हर second कतिना data आता-जाता है। 1,000 requests/second और 5KB average response वाली service को लगभग 5MB/second outbound bandwidth चाहिए।

Important Words:

- **Outbound** – बाहर जाने वाला

2. Real-World Example / Case Study

English:

Estimating for a Zomato/Swiggy-scale order system: 10 million orders per day gives average QPS ~100/sec. But dinner-rush peak traffic might be 8x average, so peak QPS ~800/second -- a number that directly motivates async processing and caching.

Hindi:

Zomato/Swiggy जैसे order system के लिए estimate करें: 1 करोड़ orders/day से average QPS ~100/sec आता है। लेकिन dinner-rush का peak traffic average से 8 गुना हो सकता है, तो peak QPS ~800/second -- यह number सीधे async processing और caching की ज़रूरत दिखाता है।

English:

For storage: if each order record is roughly 1KB and you get 10 million orders/day, that's ~10GB/day, ~3.65TB/year -- very manageable for a single well-indexed relational database.

Hindi:

Storage के लिए: अगर हर order record लगभग 1KB का है और आपको रोज़ 1 करोड़ orders मिलते हैं, तो यह ~10GB/day, ~3.65TB/year बनता है -- यह एक अच्छे से indexed single relational database के लिए बिल्कुल संभालने लायक है।

English:

A second worked example: estimating cache memory for Meesho's catalog. If 20% of 50 million products account for 80% of traffic, and each cached entry is about 2KB, caching that hot 20% needs roughly 20GB of memory.

Hindi:

एक और worked example: Meesho के catalog के लिए cache memory का अंदाज़ा। अगर 5 करोड़ products में से 20% products ही 80% traffic लाते हैं, और हर cached entry लगभग 2KB की है, तो उस hot 20% को cache करने के लिए लगभग 20GB memory चाहिए।

3. Diagram (English labels kept as-is)

Estimating QPS and storage for an order system:	
Given: 10,000,000 orders/day, ~1KB per order record	
Average QPS	= 10,000,000 / 100,000 sec =~ 100 orders/sec
Peak QPS	= Average x 8 (dinner rush) =~ 800 orders/sec
Daily storage	= 10,000,000 x 1KB = ~10 GB/day
Yearly storage	= 10 GB x 365 = ~3.65 TB/year

4. Trade-offs Table (Reference Table – English)

Estimation choice	Why it's acceptable	Risk if ignored
Using round numbers	Interviewer cares about reasoning, not precision	Wasting time on exact arithmetic
Assuming a peak-to-average ratio	Real traffic is never flat	Under-designing for the moment that breaks
Stopping at QPS/storage only	Good starting point	Missing bandwidth or memory needs

5. Common Interview Questions

Q1: "Estimate QPS and storage for a URL shortener with 100 million new URLs per month."

English:

Model approach: Convert to per-second write QPS, estimate read QPS separately (reads are far higher), estimate storage per URL, project over years, and state the read:write ratio.

Hindi:

Model approach: इसे per-second write QPS में बदलें, read QPS अलग से estimate करें (reads कहीं ज़्यादा होती हैं), हर URL की storage का अंदाज़ा लगाएं, कई सालों तक project करें, और read:write ratio बताएं।

Q2: "Why did you assume peak traffic is 5x average instead of using the average?"

English:

Model approach: Explain that systems fail at peak, not average, and real-world traffic isn't uniform across 24 hours -- citing a believable, product-specific reason makes it defensible.

Hindi:

Model approach: समझाएं कि systems average पर नहीं बल्कि peak पर fail होते हैं, और असली traffic 24 घंटे एक जैसा नहीं रहता -- एक विश्वसनीय, product से जुड़ी वजह देने से यह assumption defend करने लायक बनती है।

Important Words:

- **Defensible** – बचाव करने लायक / justify किया जा सकने वाला

Q3: "Estimate bandwidth for 50,000 concurrent viewers at 2Mbps each."

English:

Model approach: Multiply directly -- $50,000 \times 2\text{Mbps} = 100\text{ Gbps}$ total outbound bandwidth, then reason out loud about why this is exactly what a CDN is designed to absorb.

Hindi:

Model approach: सीधे multiply करें -- $50,000 \times 2\text{Mbps} = 100\text{ Gbps}$ कुल outbound bandwidth, फरि ज़ोर से बोलकर समझाएं कि यही वह load है जिससे एक CDN संभालने के लिए बना है।

6. Practice Exercises

English:

- Estimate average and peak QPS for a ride-hailing app with 5 million daily rides, assuming a 4x peak-to-average ratio.
- Estimate yearly storage for a chat app: 50 million users, 20 messages/day, 200 bytes each.
- Practice saying your estimation process out loud, narrating each step, in under 3 minutes.

Hindi:

- एक ride-hailing app के लिए, जिसमें रोज़ 50 लाख rides होती हैं, 4x peak-to-average ratio मानकर average और peak QPS निकालें।
- एक chat app के लिए yearly storage निकालें: 5 करोड़ users, हर एक 20 messages/day, हर message 200 bytes।
- अपनी estimation process को 3 मिनट से कम में ज़ोर से बोलकर, हर step बताते हुए practice करें।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"Back of the Envelope Storage Estimates System Design Interview"	Focused storage walkthroughs.
English	"Back-of-the-Envelope Calculations: Estimations for Beginners"	Beginner-paced walkthrough.

Language	Video & Channel	Why it helps
Hindi	"What is Back of Envelope calculation with examples" — Chai aur Code	Hindi walkthrough with examples.
Hindi	"Back-Of-The-Envelope Estimation Capacity Planning of Facebook"	Applies the method to a familiar system.

8. Key Takeaways

English:

- Back-of-the-envelope estimation quickly sizes a system's scale before you design its architecture.
- Method: clarify the metric, break into a formula, use round numbers, compute out loud, sanity-check.
- Always distinguish average load from peak load -- systems fail at peak.
- Being off by 2x is fine; being off by 1000x signals a fundamental misunderstanding.
- Read:write ratio, QPS, and storage estimates directly justify later decisions.
- This is a spoken skill -- practice narrating your math, not just computing it silently.

Hindi:

- Back-of-the-envelope estimation architecture design से पहले system के scale का जल्दी अंदाज़ा देती है।
- Method: metric साफ़ करें, formula में तोड़ें, round numbers use करें, ज़ोर से बोलकर calculate करें, sanity-check करें।
- Average load और peak load में हमेशा फर्क करें -- systems peak पर fail होते हैं।
- 2x गलत होना ठीक है; 1000x गलत होना एक बुनयिदी गलतफहमी दिखाता है।
- Read:write ratio, QPS, और storage estimates आगे के decisions को सीधे justify करते हैं।
- यह एक बोलने वाली skill है -- सिर्फ़ चुपचाप calculate ना करें, अपना math बोलकर बताने की practice करें।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Arithmetic	अंकगणति	Basic mathematical calculation
Defensible	बचाव करने लायक / justify किया जा सकने वाला	Able to be reasonably justified
Outbound	बाहर जाने वाला	Data going out of the system
Precision	सटीकता	Being exactly correct
Sanity-check	सैनिटी-चेक / जांच लेना	A quick check that a result seems reasonable
Sizing (figurative)	आकार का अंदाज़ा लगाना	Estimating the scale of something